

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1516

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I S Teja Yadav(192525447) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S Teja Yadav

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.
2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage.
Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment.
Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

1. Auto-Scaling and Load Balancing for Seasonal E-Commerce Traffic

An e-commerce company may experience unpredictable traffic spikes during seasonal sales such as **Black Friday, Diwali, Christmas, or a major festival sale**. During normal periods, the number of users may be moderate, but during a sale millions of customers may access the website simultaneously. If the infrastructure cannot handle the sudden increase, the website may become slow or unavailable. **Cloud auto-scaling and load balancing** can efficiently handle such unpredictable demand.

A typical architecture is:

Customers → Load Balancer → Auto-Scaling Application Servers → Database/Storage

A **load balancer** distributes incoming requests among multiple application servers. Instead of sending all users to a single server, requests are distributed across several healthy servers. For example, if an e-commerce website has ten application servers, the load balancer distributes traffic among them according to the selected balancing method. Health checks can detect failed servers and automatically remove them from the traffic pool.

Auto-scaling dynamically changes the number of servers according to workload. During normal traffic, the company may need only a few servers. When the sale starts and traffic increases, the cloud platform automatically launches additional instances. When the sale ends and traffic decreases, unnecessary instances are terminated.

For example, consider an online shopping platform such as **Amazon** during a major festival sale. Thousands or millions of customers may simultaneously search for products, add items to carts, make payments, and place orders. Auto-scaling can increase application capacity when CPU usage, request rate, or other predefined metrics cross a threshold. Load balancing then distributes the new traffic across the available instances.

This approach reduces **downtime** because there is no dependence on a single server. If one server fails, the load balancer redirects requests to healthy servers. Auto-scaling also provides additional capacity before the existing servers become overloaded.

It improves **user experience** by reducing page-loading time, request failures, and service interruptions. Customers can browse products, add items to their carts, and complete payments without experiencing significant delays.

Monitoring tools play an important role in this architecture. Monitoring systems continuously track CPU utilization, memory usage, network traffic, request rate, response time, error rate, and server health. When resource utilization increases, monitoring data can trigger auto-scaling policies. Alerts can also notify administrators about abnormal conditions.

For example:

Normal traffic → 5 servers

Sale begins → Traffic increases

Auto-scaling → 20 servers

Load balancer → Distributes traffic

Sale ends → Servers reduced to 5

Monitoring also helps administrators analyse historical traffic patterns and improve future capacity planning.

Conclusion

Cloud **auto-scaling and load balancing** allow e-commerce companies to handle unpredictable seasonal traffic efficiently. Auto-scaling provides resources according to demand, while load balancing distributes requests and removes failed servers. Monitoring provides real-time visibility and enables proactive action. Together, these technologies reduce downtime, improve performance, control costs, and provide a better customer experience during high-demand sales.

2. Secure and Durable Cloud Storage for Backup Data

Organizations generate large volumes of backup data, including databases, application files, documents, and system images. Storing all this data using traditional physical storage can be expensive and difficult to manage. **Cloud storage** provides scalable, durable, redundant, and secure storage for backup data.

Cloud storage platforms generally store data across multiple physical devices and facilities. This provides **data redundancy** and protects against hardware failures. If one storage device fails, another copy or redundant representation of the data can still be available.

Durability refers to the ability of a storage service to protect data from accidental loss or hardware failure. Cloud providers use techniques such as replication, erasure coding, checksums, and geographically distributed storage to achieve high durability.

For example, an organization can store its backup files in cloud object storage. Important backups can also be replicated to another geographic region. If the primary storage location experiences a major failure, the organization can recover data from the secondary location.

Security is also important because backup files may contain sensitive information. **Encryption** should be applied both to data at rest and data in transit. Encryption at rest protects stored backup files, while encryption during transmission protects data as it moves from the organization's systems to cloud storage.

Access control ensures that only authorized users and applications can access backup data. Role-based access control and least-privilege policies can be used. For example, a backup administrator may have permission to create and restore backups but not modify unrelated production resources. Multi-factor authentication should be used for privileged accounts.

Versioning can protect against accidental deletion or overwriting. If a backup file is modified or deleted accidentally, an earlier version can be restored.

Cloud storage also provides significant **cost advantages** compared with traditional storage systems. In a traditional datacentre, an organization must purchase storage hardware in advance and pay for maintenance, electricity, cooling, physical space, and hardware replacement. Cloud storage follows a usage-based model, allowing organizations to pay mainly for the capacity they actually use.

Cloud providers also offer different storage classes. Frequently accessed backups can be stored using standard storage, while older backups can be moved to lower-cost archival storage.

Disaster Recovery Example

Consider a bank whose main datacentre is affected by a natural disaster. Its local backup systems may also become unavailable. If encrypted backups have been replicated to cloud storage in another region, the bank can retrieve the required data and restore its applications in a disaster recovery environment.

The recovery process can be:

Production System → Automated Backup → Encrypted Cloud Storage → Secondary Region → Recovery Environment

Conclusion

Cloud storage provides **high durability, redundancy, scalability, security, and cost efficiency** for large backup datasets. Encryption protects data from unauthorized access, while IAM and access controls restrict who can access backups. Replication and geographically separated storage support disaster recovery. Compared with traditional storage, cloud storage eliminates much of the need for upfront hardware investment and allows organizations to scale storage according to their requirements.

3. CDN and Global Cloud Deployment for a Mobile Application

A mobile application serving users across different countries must provide fast response times and high availability. If all users connect to a single server located in one region, users who are geographically far away may experience high latency. A **Content Delivery Network (CDN)** solves this problem by distributing content through geographically distributed edge servers.

A typical architecture is:

Mobile Users → Nearest CDN Edge Server → Application/Origin Server

A CDN stores frequently requested content such as images, videos, JavaScript files, game assets, and other static resources at **edge locations**. When a user requests content, the CDN attempts to serve it from a nearby edge server rather than contacting the central origin server every time.

For example, suppose a gaming company has its main cloud infrastructure in the United States but has players in India. Without a CDN, every request may travel to the United States and back, increasing latency. With CDN edge servers in India or nearby regions, frequently accessed game assets can be delivered from a much closer location.

Edge servers improve user experience by reducing the physical distance between users and content. Lower latency results in faster page loading, quicker content delivery, and smoother application performance.

CDNs are particularly important for **real-time streaming applications**. A video streaming service can cache video segments at edge locations. Users receive these segments from a nearby CDN server, reducing buffering and improving playback quality.

They are also useful in gaming. Game updates, maps, images, and other large assets can be distributed through edge servers. This reduces the load on the central infrastructure and improves download speeds.

Cloud providers support global availability by operating infrastructure in multiple **regions and availability zones**. An application can be deployed in several geographic locations, and global traffic-management systems can direct users to an appropriate healthy region.

For example:

Users in India → Asia Region

Users in Europe → European Region

Users in USA → North American Region

If one region experiences an outage, traffic can potentially be redirected to another healthy region.

Cloud providers also offer global load balancing, DNS-based traffic management, replicated storage, and multi-region database services. These technologies help maintain availability for globally distributed users.

Real-Time Streaming Example

Consider a live sports streaming application with millions of users worldwide. During a major match, a large number of users request the same video content. CDN edge servers cache and deliver video segments close to users. The origin servers therefore receive fewer direct requests.

The architecture becomes:

Users → Global Traffic Manager → CDN Edge → Streaming Origin → Cloud Storage

This reduces latency, bandwidth usage, and origin-server load.

Conclusion

CDNs improve application performance by delivering content from **nearby edge servers**, reducing latency and network distance. Cloud providers support global availability through multiple regions, availability zones, global load balancing, and replicated services. For streaming and gaming applications, CDN and edge computing are especially valuable because they provide faster content delivery, reduced buffering, and a better experience for users across different regions.

4. Cloud Compliance and Security for a Financial Institution

Financial institutions handle highly sensitive information such as customer accounts, payment details, credit information, and transaction records. Therefore, they must follow strict security and regulatory requirements when using cloud services. Cloud providers support compliance through security controls, auditing capabilities, encryption, logging, monitoring, and compliance certifications.

Cloud providers typically provide tools that help organizations meet requirements related to security, data protection, access management, auditing, and regulatory compliance. However, the organization is still responsible for properly configuring its cloud environment.

Logging is essential for auditing. Cloud systems can record activities such as user logins, API requests, data access, configuration changes, and administrative operations. These logs provide evidence of who performed an action and when it occurred.

For example, if an employee accesses a sensitive financial transaction database, the system should record the identity, time, resource accessed, and action performed. Logs can help detect unauthorized activity and support investigations.

Monitoring provides continuous visibility into the security and performance of cloud resources. Monitoring systems can detect unusual login patterns, repeated failed authentication attempts, unexpected data access, high network traffic, or unauthorized configuration changes. Automated alerts can notify security teams when suspicious activity is detected.

Encryption protects sensitive data from unauthorized access. Data should be encrypted both **at rest** and **in transit**. Databases, backups, and stored files should be encrypted. Communication between applications and cloud services should also use secure encrypted connections.

Identity and access management is equally important. Financial institutions should use **multi-factor authentication, role-based access control, least privilege, and privileged-access management**. Employees should receive only the permissions required for their jobs.

Data Sovereignty

Data sovereignty means that data is subject to the laws and regulations of the country or jurisdiction where it is stored or processed. A financial organization may be required to keep certain customer information within a particular country.

The company should therefore select appropriate cloud regions and verify where data is physically stored. It should also understand where backups and replicas are located. Cloud policies can be configured to prevent sensitive data from being moved to unauthorized regions.

Financial Transaction Example

Consider a bank processing an online money transfer. A customer transfers ₹1,00,000 from one account to another.

The transaction data should be encrypted during transmission and stored in encrypted databases. IAM ensures that only authorized banking applications can access the transaction system. Logging records the transaction-related system activity, while monitoring detects suspicious behaviour such as unusual access or repeated failed authentication attempts.

Audit teams can later examine the logs to verify compliance and investigate security incidents.

Conclusion

Cloud providers support financial institutions through **encryption, IAM, logging, monitoring, auditing tools, security controls, and compliance capabilities**. Organizations must properly configure these services and enforce security policies. Data sovereignty can be maintained by selecting approved regions and controlling where data and backups are stored. These measures help financial institutions protect sensitive transactions while meeting regulatory and security requirements.

5. Containers, Virtual Machines, and Kubernetes

Containerization is a technology used to package an application together with its dependencies so that it can run consistently across different environments. Containers are different from virtual machines (VMs) in terms of architecture, performance, and resource usage.

A virtual machine runs a complete **guest operating system** on top of a hypervisor. Each VM has its own operating system, libraries, and application.

The architecture is:

Hardware → Hypervisor → VM → Guest OS → Application

Containers share the host operating system kernel while isolating individual applications.

The architecture is:

Hardware → Host OS → Container Engine → Containers → Applications

Because containers do not require a separate guest operating system for each application, they are generally **lighter and faster to start** than VMs. They require less memory and storage and allow more application instances to run on the same physical machine.

VMs provide stronger isolation at the hardware virtualization level and are useful when applications require separate operating systems. Containers provide efficient resource utilization and are particularly suitable for microservices and cloud-native applications.

Managing a large number of containers manually can become difficult. **Container orchestration platforms such as Kubernetes** simplify this process. Kubernetes can automatically deploy containers, schedule workloads, restart failed containers, perform health checks, provide service discovery, and scale applications.

For example, if a web application normally runs five containers but traffic increases, Kubernetes can automatically increase the number to twenty. When traffic decreases, it can reduce the number again.

Kubernetes also supports **rolling updates**, allowing new application versions to be deployed gradually without taking the entire service offline. If a container fails, Kubernetes can automatically create a replacement.

Portability

Containers provide excellent portability because the same container image can run on different cloud platforms and on-premises systems, provided the required container runtime/orchestration environment is available.

For example:

Developer Laptop → Testing Environment → Cloud A → Cloud B

The application can use the same container image in each environment. This reduces the problem of differences between development and production environments and can reduce cloud vendor lock-in.

Microservices Example

Consider an online shopping application divided into multiple microservices:

- User Service
- Product Service
- Cart Service
- Payment Service
- Order Service
- Notification Service

Each service can run in its own container. Kubernetes manages these containers and allows each service to scale independently.

For example, during a sale, the **Product Service** may receive much more traffic than the Notification Service. Kubernetes can increase Product Service containers without unnecessarily increasing the others.

Conclusion

Containers are generally lighter, faster, and more resource-efficient than virtual machines because they share the host operating system kernel. Kubernetes simplifies container deployment, service discovery, health management, updates, and scaling. Container portability allows applications to move between different cloud providers and on-premises environments. Therefore, containers and Kubernetes are highly suitable for modern **microservices-based cloud applications**.